Lecture 4: LLM Agents and Finance Applications

Large Language Models in Finance

Juan F. Imbet

Paris Dauphine – PSL University

Outline

Why Passivity is a Limitation

The static LLM

A language model maps a prompt to a probability distribution over tokens.
It is **stateless**: each call is independent; it cannot change its environment.

A portfolio manager asks:

> *"Given Apple's last earnings call and the most recent 10-K, what are the key risks to revenue growth vs. two years ago?"*

Problems with a plain LLM:

- Documents may not fit in context
- Cross-referencing two sources is unreliable
- A follow-up ("now compare to Microsoft") restarts from scratch

From Static Oracle to Agent

Recall the portfolio manager's question: compare revenue risks across Apple's earnings call and 10-K, then follow up with Microsoft. A plain LLM cannot fit the documents, cross-reference them reliably, or carry context forward.

THE BIGGER PICTURE

An agent embeds the LLM inside a loop: observe, reason, act, incorporate result. Instead of a one-shot oracle, you get a system that can fetch the right documents, cross-reference them, and carry context across follow-up questions — the behaviour the portfolio manager actually needs.

The Perceive–Reason–Act Loop

UNDER THE HOOD

A PRA loop is a triple (π, μ, ϕ):

- $\phi : \mathcal{O} \times \mathcal{M} \to \mathcal{M}$ **memory update** — observation + memory → new memory
- $\pi : \mathcal{M} \to \Delta(\mathcal{A})$ **policy** — memory → distribution over actions
- $\mu : \mathcal{A} \times \mathcal{E} \to \mathcal{O}$ **environment response** — action → new observation

In an LLM agent:

- π is the language model
- Memory = the structured prompt (history, tool results, system instructions)
- Action = text response, tool call, or termination
- Observation = tool output, retrieved document, user message

Key bottleneck

Memory is bounded by the context window (≈100 pages for GPT-4-class)

ReAct: Interleaving Thought and Action

UNDER THE HOOD

ReAct (Yao et al., 2022) interleaves reasoning traces and actions:

$$\tau = (T_1, A_1, O_1,\ T_2, A_2, O_2,\ \ldots,\ T_n, A_n, O_n)$$

T_i = natural-language thought $\quad$ A_i = action $\quad$ O_i = observation

Why explicit reasoning helps:
Computing $f(t)$ = closing price of AAPL on date t:

- CoT-only: model approximates $\hat{f}(t)$ from training patterns — unreliable
- ReAct: model calls `market_data_api(date=t)`, receives exact value

Downstream reasoning uses the precise number, not the approximation.

Practical result

ReAct reduces hallucination by grounding each reasoning step in tool-call observations.

ReAct in Action: Filing Retrieval

Query: "What was Tesla's stated liquidity risk policy in its most recent 10-K?"

Thought 1	I need Tesla's most recent 10-K. Use the EDGAR search tool.
Action 1	`edgar_search(ticker="TSLA", form="10-K", limit=1)`
Obs 1	`{accession: "0001318605-24-000004", filed: "2024-01-26"}`
Thought 2	I have the accession. Retrieve the liquidity section specifically.
Action 2	`edgar_section(accession="...", section="Liquidity")`
Obs 2	`[3,200 words of text...]`
Thought 3	I have the relevant section. Extract the stated policy.
Action 3	FINISH(structured summary of liquidity policy)

Three tool-call steps replace one hallucination-prone generation step.

Beyond ReAct: Planning Architectures

Tree-of-Thought (ToT): structures reasoning as a search tree; evaluates multiple candidate paths before committing.

Reflexion (Shinn et al., 2023): adds verbal self-critique after each trajectory.

$$m_0^{(k+1)} \leftarrow \ell^{(k)} \oplus m_0^{(k)}$$

$\ell^{(k)}$ = lesson from episode k; prepended to memory on next episode.

Finance application: an agent identifying comparable companies might generate the lesson:
"I should filter by market cap and GICS sub-industry simultaneously, not sequentially."
This carries forward without retraining.

Key insight

Verbal self-improvement substitutes for expensive fine-tuning in low-data regimes.

Three Types of Agent Memory

Type	Size	Latency	Persistence
In-context	$\leq$ ctx window	≈ 0	Session only
Retrieval-augmented	Unbounded	10–100 ms	Persistent
Long-term (DB)	Unbounded	1–10 ms	Persistent

In-context: current conversation, recent tool results. Zero latency. Discarded at session end.
Retrieval-augmented: vector database; approximate nearest-neighbour search. Suitable for semantic queries over large corpora (10,000 SEC filings).
Long-term: relational or graph DB; precise key-value lookups. Suitable for entity metadata, fiscal calendar, analyst revision history.

Financial agents layer all three: right memory tier for each information need.

Retrieval Error Propagates into Reasoning

Given N document vectors $\{\mathbf{v}_i\} \subset \mathbb{R}^D$ and query $\mathbf{q}$:

$$\text{Retrieve}(q, k) = \text{argtopk}_i \ \frac{\mathbf{q} \cdot \mathbf{v}_i}{\|\mathbf{q}\| \, \|\mathbf{v}_i\|}$$

Two failure modes:

- **Missed relevant doc** $\rightarrow$ hallucinated gap in the answer
- **Spurious retrieval** $\rightarrow$ noise introduced into reasoning

In finance, retrieval errors matter

A faithfulness failure in a compliance summary can be a regulatory violation. Faithfulness $< 0.85 \rightarrow$ trigger human review, not automated publication.

The Tool Use Paradigm

A **tool** is a triple $\mathcal{T} = (\textit{name}, \textit{schema}, f)$:

- *name* — unique identifier
- *schema* — JSON Schema: input domain and output co-domain
- $f : \mathcal{D} \to \mathcal{R} \cup \{\textsc{Error}\}$ — implementation

Highest-leverage investment in agent reliability

Clear, precise field descriptions — specifying units, valid ranges, edge-case behaviour. These descriptions propagate into the JSON Schema the model receives.

The Tool Use Paradigm: Mechanics

UNDER THE HOOD

How it works (OpenAI/Anthropic/Google):

1. Tools declared in system prompt as JSON array
2. Model generates structured tool-call message with function name + arguments
3. Execution environment dispatches function, returns result as new message
4. Model continues reasoning with tool result in context

The model never executes code itself: it emits a structured request, and a trusted execution environment runs f and returns the result as a new observation. This separation is what makes tool use auditable and safe to restrict — the agent's action space is exactly the set of declared tools.

Custom Tool: DCF Calculator

```
from pydantic import BaseModel, Field

class DCFInput(BaseModel):
    free_cash_flows: list[float] = Field(
        description="Projected FCFs in USD millions,
            chronological"
    )
    wacc: float = Field(
        description="WACC as a decimal (0.08 = 8%)", ge=0.0,
            le=1.0
    )
    terminal_growth_rate: float = Field(
        description="Perpetual growth rate as a decimal", ge
            =0.0, le=0.15
    )
```

- Pydantic field descriptions propagate into the JSON Schema
- Validation errors returned to LLM as structured feedback — agent regenerates with corrected arguments

Tool Reliability and Error Recovery

Production agents must handle tool failures gracefully. Failure sources: network timeouts, API rate limits, schema validation errors, empty results, ambiguous queries.

Layered recovery strategy:

1. **Innermost**: retry with exponential back-off (transient failures)
2. **Middle**: interpret error as observation; generate alternative action (different endpoint, relaxed date range, decomposed query)
3. **Outermost**: $\textsc{RequestHumanHelp}$(description) rather than returning a hallucinated answer

Never pass errors silently

A robust tool calling protocol always returns an informative $\textsc{Error}(\textsc{msg})$. The agent policy treats this as a valid observation and generates a recovery action.

Skills and Hooks

Skill: a named, parameterised agent sub-workflow $\mathcal{S} : \mathcal{P} \times \mathcal{M}_{\text{in}} \to \mathcal{M}_{\text{out}}$. Abstracts over prompt engineering, tool selection, and error handling for a specific task.

Examples: `SummariseEarningsCall`, `RetrieveRiskFactors`, `GenerateCompsTable`. Skills compose: a coordinator invokes them in sequence without knowing their internals.

Hook: a callback function $(\mathcal{E}, h)$ where $\mathcal{E}$ is an event type and $h : \mathcal{E} \times \mathcal{M} \to \mathcal{M}$ is a handler.

Financial hooks:

- **Audit hook**: logs every tool call to an immutable audit trail (MiFID II)
- **Compliance hook**: checks proposed response before any FINISH
- **Cost hook**: accumulates token usage; alerts on budget threshold breach

Hooks decouple cross-cutting concerns from the agent's core reasoning loop.

Multi-Agent Systems and Role Delegation

A multi-agent system is $(\mathcal{AG}, \mathcal{C}, \delta)$: agents, communication topology, routing function.

Key insight: task decomposition reduces error — a specialist agent handling a narrow subtask outperforms a generalist managing everything.

Hierarchical delegation for equity research:

1. *Coordinator* — decomposes request, assembles final output
2. *Data Agent* — retrieves financial statements and market data
3. *Quant Agent* — computes valuation ratios, scenario analysis
4. *Analyst Agent* — drafts qualitative commentary
5. *Compliance Agent* — checks regulatory compliance, appends disclosures

Production result

First-draft research report in ≈ 4 minutes vs. 2 days for a human team. Humans review compliance flags and quant scenario assumptions.

RAG: Theory and Finance Motivation

THE BIGGER PICTURE

Motivation: a large asset manager's research library has 50,000 documents. No context window holds this; no fine-tuning reliably memorises it. The way out is to *retrieve* the handful of relevant passages at query time and let the model reason over those — knowledge stays external, current, and auditable.

Retrieval-Augmented Generation (Lewis et al., 2020):

$$p(a \mid q) = \sum_{z \in \mathcal{D}} p(z \mid q)\, p(a \mid q, z)$$

In practice: approximate by conditioning on top-k retrieved documents only. Knowledge stays external, current, and auditable.

RAG: Deployment and Metadata Filtering

Dominant deployment (naive RAG):

- Frozen bi-encoder retriever (sentence transformer)
- Closed-source LLM generator via API
- Sacrifices end-to-end optimisation; dramatically reduces cost

Metadata filtering is critical

Query for "Apple CFO supply chain" should be restricted to Apple documents. Metadata filtering reduces irrelevant retrievals by $\approx 60\%$.

RAG Evaluation: RAGAS Metrics

Metric	Definition
Faithfulness	Fraction of answer claims supported by retrieved context
Answer Relevance	Semantic similarity between answer and query
Context Precision	Fraction of retrieved chunks relevant to query
Context Recall	Fraction of ground-truth claims covered by context

Finance priority order:

1. **Faithfulness** is most critical: unfaithful answer = hallucinated claim = compliance failure when presented to clients. Faithfulness $< 0.85 \rightarrow$ block automated publication; trigger human review.
2. **Context Recall** matters for completeness: high faithfulness + low recall = accurate but incomplete (material disclosures may be omitted).

Earnings Call Analysis Pipeline

Five stages:

1. **Ingestion**: transcript from vendor API or EDGAR
2. **Segmentation**: classify speaker turns as guidance / analyst Q / clarification
3. **Metric extraction**: identify numerical guidance; compare to prior and consensus
4. **Sentiment**: directional score per segment (financially calibrated scale)
5. **Synthesis**: structured summary with transcript citations

Predictive evidence: Kim et al. (2024) — GPT-4 fed anonymised financial statements predicts earnings direction at 60% accuracy vs. 53% for human analysts.

Domain knowledge matters

General models misinterpret financial language: "margin compression was manageable" = negative signal. "exceptional inventory build" (consumer electronics) = negative signal.

Report Generation and Filing Search

Automated report generation — core principle:

- Prompting "write a report about NVIDIA" → hallucinated figures
- Data agent assembles *verified* data package first
- Writer agent produces prose *describing* that package, citing each figure by source
- MiFID II substantiation requirements apply to algorithmic research too

---

Filing Q&A — citation enforcement wrapper:

- Every claim must be accompanied by a chunk citation
- Cited chunk must contain the claimed information
- Unsupported claims → `[Claim not verified; human review required]`
- Two-model pipeline: large generator + smaller verifier model

FinanceBench benchmark: GPT-4-Turbo with retrieval incorrectly answers or refuses **81%** of questions — production-grade financial RAG is

Human-in-the-Loop Design Patterns

Pattern	**Timing**	**Appropriate for**
Pre-execution approval	Before any action	Trade execution, client comm
Checkpoint review	At predefined gates	Report generation, monitorin
Post-hoc audit	After completion	News classification, filing ale

Match the pattern to **reversibility and consequence**:

- Irreversible + high-consequence $\rightarrow$ pre-execution approval
- Reversible + medium-consequence $\rightarrow$ checkpoint review
- Low-stakes + high-volume $\rightarrow$ post-hoc audit

Audit Trails and Explainability

An audit trail $\mathcal{A} = (r_1, \ldots, r_T)$ where each record r_t contains: timestamp, record type, full input/output payload, and:

$$\text{hash}_t = H(\text{payload}_t \,\|\, \text{hash}_{t-1})$$

Chained hashes provide **tamper evidence**: modifying any historical record invalidates all subsequent hashes.

Regulatory requirements: MiFID II Article 25, SEC Rule 17a-4 apply to algorithmically generated decisions influencing investment outcomes.

Explainability vs. auditability:

- Audit trail records *what* the agent did
- Explanation addresses *why*
- For ReAct agents: the reasoning trace (THOUGHT steps) *is* the explanation
- Strong argument for agents that externalise reasoning in natural language

Prompt Injection and Adversarial Robustness

Prompt injection: malicious instructions in tool outputs or retrieved documents override the system prompt.

Indirect injection (Greshake et al., 2023): attacker controls a web page; embeds hidden instructions executed by any agent retrieving that page.

Defence in depth (no single control sufficient):

1. **Input sanitisation**: strip instruction-like patterns from retrieved text
2. **Privilege separation**: trusted (system prompt) vs. untrusted (retrieved docs)
3. **Invariant monitoring**: separate agent blocks actions violating invariants
4. **Dual-key authorisation**: consequential actions require human + agent confirmation

Formal model: confused deputy problem

A powerful program (agent) is manipulated by a less-privileged input (document) into exercising elevated privileges. Defence: capability-based security — trade-execution API key stored separately, requiring additional

Summary: The Agent Stack

Capability	**Key mechanism**
Basic LLM	Prompt $\rightarrow$ token distribution (stateless)
Agent loop	Perceive–Reason–Act; ReAct interleaving
Extended memory	Vector database; hybrid search; chunking
External tools	Function calling; Pydantic validation
Composition	Skills; hooks; multi-agent delegation
Finance applications	Earnings analysis; report generation; Q&A
Safety	HITL; audit trails; injection defences

Connection to Chapter 5

Chapter 5 applies these agent architectures to business valuation — a task requiring document retrieval, numerical computation, structured synthesis, and human oversight: exactly the stack developed here.

Appendix

Extra Material: Frameworks, Hybrid Search, Chunking

Extra / optional material — beyond the core lecture.

Orchestration Frameworks

Framework	Design philosophy	Best for
LangChain	Composable chains	Diverse data sources
LlamaIndex	Data-intensive retrieval	Large document corpora
AutoGen	Multi-agent conversation	Team-structured workflows

Auditability: AutoGen serialises full conversation histories — natural audit log satisfying MiFID II best-execution documentation requirements.

LangChain and LlamaIndex require explicit configuration for persistence. For compliance-sensitive applications, choose the framework that fits *both* task type and regulatory audit trail requirements.

Hybrid Search: Dense + Sparse

Dense retrieval (embedding-based): captures semantic similarity. Fails on exact match queries (CUSIP numbers, regulatory citations).

BM25 (sparse): strong on exact term matching. Defined as:

$$\text{BM25}(q, d) = \sum_i \text{IDF}(q_i) \cdot \frac{f(q_i, d)(k_1 + 1)}{f(q_i, d) + k_1(1 - b + b\,|d|/\text{avgdl})}$$

Hybrid score (after min-max normalisation):

$$\text{score}_{\text{hybrid}} = \alpha \cdot \widetilde{\text{cosine}} + (1 - \alpha) \cdot \widetilde{\text{BM25}}$$

Empirical guidance:

- $\alpha \approx 0.7$ (favour dense): QA over long financial documents
- $\alpha \approx 0.3$ (favour sparse): keyword-heavy regulatory lookup
- Reciprocal Rank Fusion (RRF) avoids normalisation entirely

Chunking Strategies for Financial Documents

A 10-K may be 200 pages with hierarchical structure. Naive fixed-size chunking severs meaning at arbitrary points.

Three strategies:

1. **Fixed-size**: L tokens, δ overlap. Simple; ignores structure.
2. **Semantic**: sentence/paragraph boundaries; EDGAR item structure for SEC filings.
3. **Hierarchical**: multi-level index (document $\rightarrow$ section $\rightarrow$ paragraph). Query first retrieves section, then paragraph within section.

Recommended for 10-K filings: hierarchical combining EDGAR items (top level) with sentence-boundary chunking within items (max 512 tokens, 64-token overlap).

Benchmark result

+15–20% on downstream QA vs. fixed-size chunking. Apple 2023 10-K: 94% paragraph recall on a manually labelled evaluation set.